

Lab: AES Lab Using OpenSSL

Estimated Time: 30 minutes

Overview

In this lab, you'll learn how to encrypt and decrypt files using Advanced Encryption Standard (AES) with OpenSSL. The lab covers the generation of an AES key, encryption of a file, and decryption to verify the process.

Learning objectives

By the end of this lab, you will be able to:

- Generate an AES encryption key
- Encrypt and decrypt files using AES with OpenSSL
- Identify the role of symmetric encryption in securing data

Prerequisites (Optional)

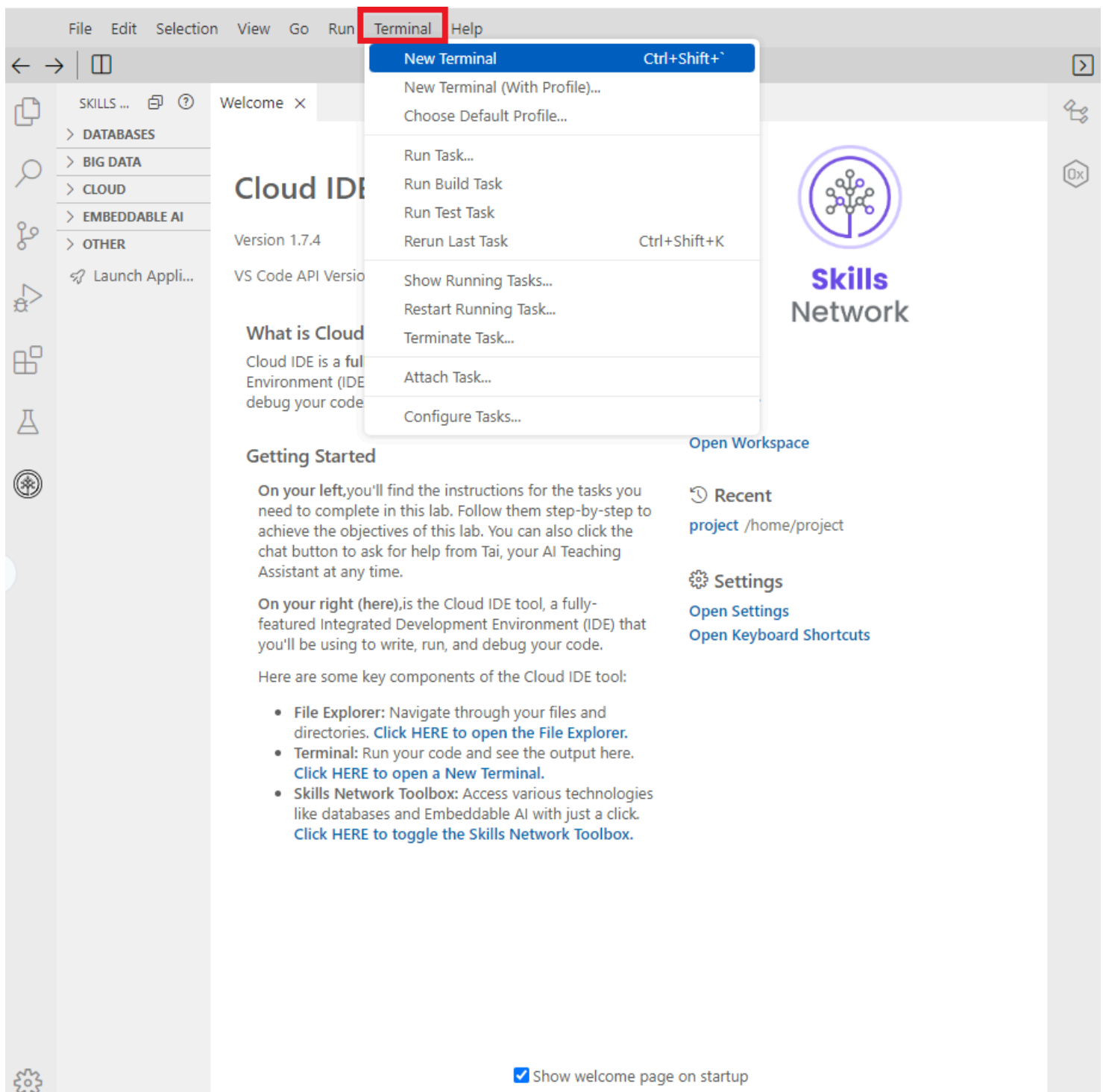
You should:

- Be familiar with the Linux command line
- Have basic understanding of encryption concepts.

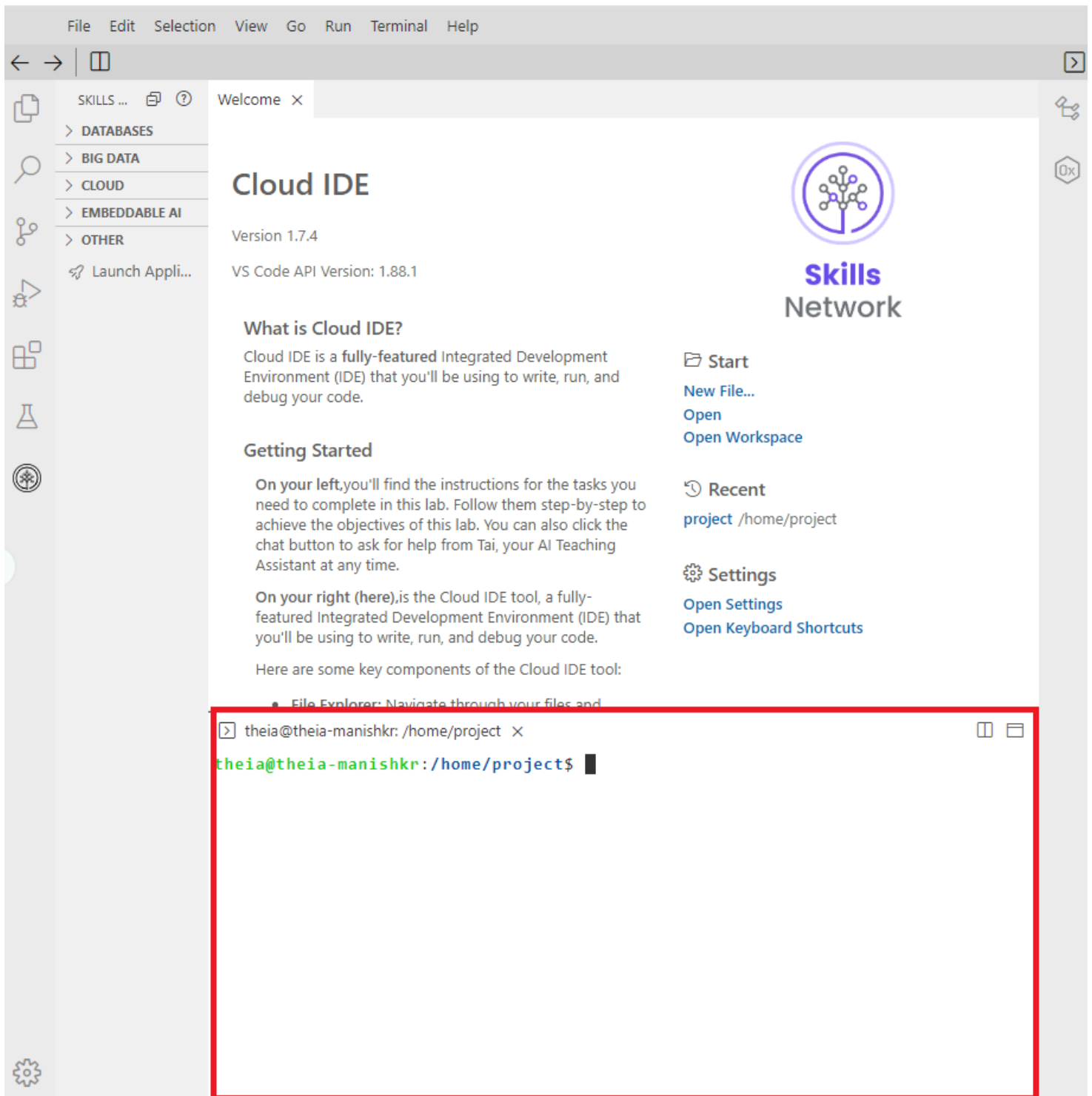
Initializing the Lab Environment

Follow the below steps to set the lab environment.

1. Select **Terminal** and select **New Terminal** to open new terminal.



2. Copy and paste the commands into the terminal for the remaining steps.



Step 1: Generate an AES Encryption Key

Enter below command to use OpenSSL to create a 256-bit AES key.

```
openssl rand -base64 32 > aes_key.txt
```

Command breakdown:

- `openssl rand`: Generates random data.
- `-base64`: Encodes the output in Base64 format.
- `32`: Specifies the key length (32 bytes = 256 bits).
- `> aes_key.txt`: Saves the key to a file.

Enter the below command to verify the key generation.

```
cat aes_key.txt
```

Step 2: Create a File to Encrypt

Enter the below command to create a file for the encryption.

```
echo "This is a test file for AES encryption." > plaintext.txt
```

Command breakdown:

- `echo`: Outputs the specified text.
- `> plaintext.txt`: Redirects the text into a file named `plaintext.txt`.

Enter the below command to verify the file content.

```
cat plaintext.txt
```

Step 3: Encrypt the File

Enter the below command to encrypt the file.

```
openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted_file.bin -pass file:aes_key.txt
```

Command breakdown:

- `openssl enc`: Performs encryption/decryption.
- `-aes-256-cbc`: Specifies AES with a 256-bit key in CBC mode.
- `-salt`: Adds a salt to enhance security.
- `-in plaintext.txt`: Specifies the input file.
- `-out encrypted_file.bin`: Specifies the output file.
- `-pass file:aes_key.txt`: Supplies the encryption key from `aes_key.txt`.

Enter the below command to verify the encrypted file.

```
cat encrypted_file.bin
```

The output will be unreadable binary data, hence confirming encryption.

Step 4: Decrypt the File

Enter the below command to decrypt the file using the same AES key.

```
openssl enc -aes-256-cbc -d -in encrypted_file.bin -out decrypted_file.txt -pass file:aes_key.txt
```

Command breakdown:

- `-d`: Specifies decryption.
- Other options are similar to the encryption command.

Enter the below command to verify the decrypted file.

```
cat decrypted_file.txt
```

You can see the original text.
This is a test file for AES encryption.

Exercises

Exercise 1: Experiment with Different Key Sizes

Objective:

Understand how different key sizes (128, 192, and 256 bits) affect the encryption and decryption.

Task:

1. Encrypt a file named `plaintext.txt` using AES with the following key sizes: 128-bit, 192-bit, and 256-bit.
2. Decrypt the files and verify that the output matches the original plaintext.
3. Compare the time taken for each encryption.

Hint:

- Use the `-aes-128-cbc`, `-aes-192-cbc`, and `-aes-256-cbc` options in OpenSSL.
- Use the `time` command to measure the duration of the encryption process.
For example:

```
time openssl enc -aes-128-cbc -in plaintext.txt -out ciphertext_128.enc -k your_password
```

- Use `sha256sum plaintext.txt decrypted.txt` to confirm that the decrypted file matches the original plaintext.

Exercise 2: Experiment with Initialization Vectors (IVs)

Objective:

Understand the role of IVs in ensuring unique ciphertexts for identical plaintexts.

Task:

1. Encrypt the same plaintext file (`plaintext.txt`) for multiple times using the same key with different IVs.
2. Compare the resulting ciphertexts to observe the differences.
3. Decrypt the ciphertexts and verify the plaintext that remains consistent.

Hint:

- Generate random IVs using OpenSSL:

```
openssl rand -hex 16
```

- Use the `-iv` option to specify different IVs during encryption.
- Compare the ciphertexts using `diff` to observe variations caused by different IVs:

```
diff ciphertext1.enc ciphertext2.enc
```

- Ensure decryption uses the correct IV to recover the original plaintext.
-

Conclusion

In this lab, you learned how to use OpenSSL to encrypt and decrypt files with AES, a widely used symmetric encryption algorithm. By generating an AES key and applying it to encrypt and decrypt files, you gained practical experience with securing data. These skills are fundamental in ensuring data confidentiality in various real-world applications.

Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.